

CREATION DES API REST EN SPRING BOOT

1. Introduction

Java Spring Framework (Spring Framework) est une infrastructure open source d'entreprise couramment utilisée qui permet de créer des applications autonomes de production qui fonctionnent sur la machine virtuelle Java (JVM). Java Spring Boot (Spring Boot) est une extension du Spring Framework pour simplifier le développement d'applications Web et de microservices avec Spring Framework grâce à trois fonctionnalités principales :

1. Configuration automatique : signifie que les applications avec les Spring Beans sont initialisées avec des dépendances prédéfinies dans les classes individuelles. Elle permet de **configurer automatiquement** votre application à partir des JAR trouvés dans votre classpath et non pas à partir des fichiers config XML
2. Simplification de la construction de l'application en ajoutant et configurant les dépendances de démarrage (Starters), en fonction des besoins de votre projet. Par exemple, la dépendance de démarrage « Spring Web » vous permet de créer des applications Web Spring avec une configuration minimale en ajoutant à votre projet toutes les dépendances nécessaires, telles que le serveur Web Apache Tomcat. Spring Security » est une autre dépendance de démarrage couramment utilisée qui ajoute automatiquement des fonctions d'authentification et de contrôle d'accès à votre application.

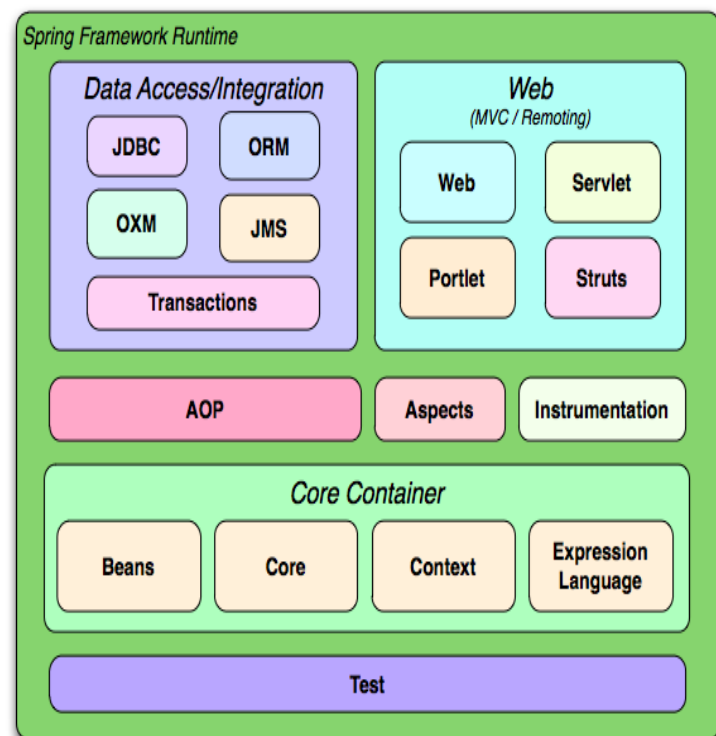
Spring Boot comprend plus de 50 Spring Starters, et de nombreux autres starters tiers sont disponibles.

3. Possibilité de créer des applications autonomes qui s'exécutent seules, sans dépendre d'un serveur Web externe, en intégrant un serveur Web embarqué tel que Tomcat ou Netty dans votre application au cours du processus d'initialisation, ce qui simplifie le déploiement d'applications sur n'importe quelle plateforme.

Architecture Spring Boot

Le framework Spring comprend des fonctionnalités organisées en environ 20 modules à savoir:

- **Spring JDBC** : utilisé pour manipuler une base de données SGBDR
- **Spring MVC** : Principalement utilisé pour développer une application Web.
- **Spring ORM** : un module couvrant de nombreuses technologies de persistance, notamment JPA, JDO, Hibernate et iBatis..
- **Spring AOP** : La programmation orientée aspect (AOP) complète la programmation orientée objet (POO).
- **Spring JMS** : JMS (Java Message Service) est un middleware utilisé pour envoyer des messages entre clients en envoyant des messages vers une file d'attente de messages, qui sont ensuite utilisés chaque fois que possible pour exécuter une transaction.
- **Spring Test**
- **Spring Expression Language (SPeL)**



Parmi ces modules, il y a les modules fondamentaux qui font partie du noyau du Spring Framework=conteneur de base (core) :

- **Core** : module contenant les classes fondamentales utilisées par tous les autres modules (équivalent de java.lang)
- **Beans** : ce module permet de manipuler les objets Java et de créer des beans

- **Context** : ce module introduit la notion de contexte d'application permettant d'enregistrer les objets et d'y accéder

Spring Boot fournit des dépendances de type starters permettant d'importer un groupe de jars pour un besoin donné.

Par exemple, le starter **spring-boot-starter-data-jpa** va vous apporter différents JAR pour utiliser Spring et JPA, afin de communiquer avec une base de données.

Si on a besoin de développer une application web, vous aurez besoin d'un **spring-boot-starter-web** pour obtenir les dépendances suivantes :

- **Jackson** : permet de parser JSON et faire le lien entre les classes Java et le contenu JSON.
- **Tomcat** : intégré, va nous permettre de lancer notre application en exécutant tout simplement le JAR sans avoir à le déployer dans un serveur d'application.
- **Hibernate** : facilite la gestion des données.
- **Logging** : remonte ce qui se passe dans l'application grâce à logback et autres.

Avec Spring Boot, vous allez tout simplement avoir une seule dépendance dans votre pom.xml :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Tous les starters sont préfixés par "spring-boot-starter". Voici quelques exemples de starters :

- spring-boot-starter-core ;
- spring-boot-starter-data-jpa ;
- spring-boot-starter-security ;
- spring-boot-starter-test ;
- spring-boot-starter-web.

L'autre avantage de starter est la **gestion des versions**. Il vous suffit d'ajouter une simple dépendance au starter de votre choix. Cette dépendance va alors ajouter, à son tour, les éléments dont elle dépend, avec les bonnes versions.

Ces starters vont charger les dépendances présentes dans le pom suivant :

```
<?xml version="1.0" encoding="UTF-8"?>
<project
  xmlns="http://maven.apache.org/POM/4.0.0"
```

```
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
    <modelVersion>4.0.0</modelVersion>
    <parent>
```

```
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-parent</artifactId>
```

```
      <version>2.6.0</version>
```

```
      <relativePath/> <!-- lookup parent from
```

```
repository -->
```

```
    </parent>
```

```
    <groupId>com.example</groupId>
```

```
    <artifactId>demo</artifactId>
```

```
    <version>0.0.1-SNAPSHOT</version>
```

```
    <name>demo</name>
```

```
    <description>Demo project for Spring
    Boot</description>
```

```
    <properties>
```

```
      <java.version>11</java.version>
```

```
    </properties>
```

```
    <dependencies>
```

```
      <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
```

```
      </dependency>
```

```
    </dependencies>
```

```
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
```

```
  </dependency>
```

```
  </dependency>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-
```

```
test</artifactId>
    <scope>test</scope>
```

```
  </dependency>
```

```
  </dependencies>
```

```
  <build>
```

```
    <plugins>
```

```
      <plugin>
```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-
```

```
plugin</artifactId>
```

```
      </plugin>
```

```
    </plugins>
```

```
  </build>
```

```
</project>
```

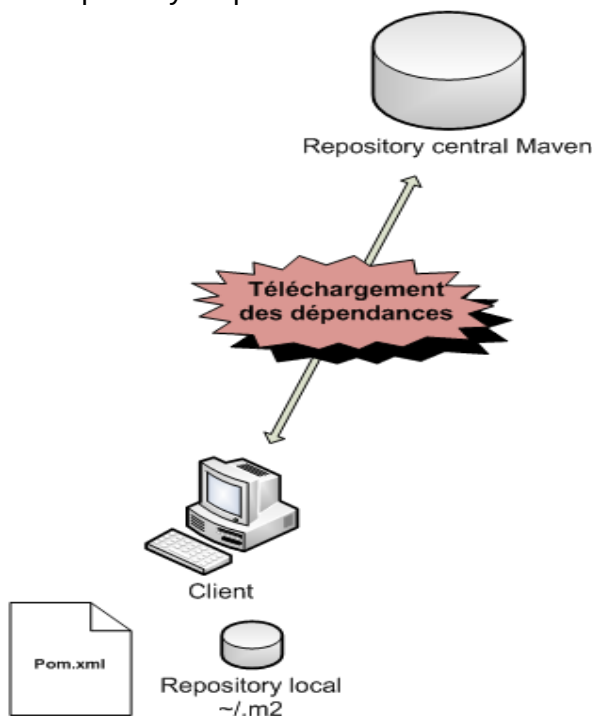
Le spring-boot-starter-parent ajouté en haut du pom.xml, permet à votre pom d'hériter des propriétés d'un autre pom et permet d'ajouter des dépendances sans indiquer leur version. En d'autres termes, ce pom.xml hérite du parent spring-boot-starter-parent qui nous permet de ne plus nous soucier des versions des dépendances ni de leur compatibilité :

En résumé

- Spring Boot est un framework qui permet de démarrer rapidement le développement d'applications ou services, en fournissant les dépendances nécessaires et en autoconfigurant celles-ci.
- Les starters permettent d'importer un ensemble de dépendances selon la nature de l'application à développer, afin de démarrer rapidement.

2. Installation et configuration de Spring Boot

Spring Boot est compatible avec Apache Maven 3.6.3 ou version ultérieure. Par défaut, Maven est un outil permettant de télécharger les librairies/dépendances sur les repositories officiels Maven (par exemple <http://repo1.maven.org/maven2>). Les dépendances à télécharger sont précisées dans le fichier pom.xml du projet. Les dépendances téléchargées sont stockées dans le répertoire ~/.m2/repository du poste client :



REPOSITORY

C'est un espace permettant de stocker tous les artefacts qui représentent:

- les projets (binaires, code source, documentation...),
- les librairies utilisées dans nos projets.

Maven définit deux types de repository :

- SNAPSHOT REPOSITORY : on y stocke les versions SNAPSHOT d'un projet "**en cours de développement**". Par exemple, si la version du projet web.com.netapsys:foo est **5.3-SNAPSHOT**, mvn deploy produira un fichier foo-5.3-SNAPSHOT.war
- RELEASE REPOSITORY : on y stocke les versions non-SNAPSHOT d'un projet dans un état stable (la version de publication du logiciel) et qu'il peut être utilisé hors développement.

Emplacement du repository local par défaut :

- Sous Windows : C:\Users\{compte-windows}\.m2 ou dans votre dossier "Mes Documents" \.m2
- Sous Linux : ~/.m2

La configuration du chemin du dépôt local se fait dans le fichier **/conf/settings.xml** du sous-répertoire .m2 contenu dans le répertoire home de l'utilisateur.

Vous pouvez changer le chemin absolu du dépôt local en ajoutant la balise **localRepository** dans le fichier config:

<localRepository>/chemin/absolu/vers/le/nouvea
u/repertoire**</localRepository>**

Exemple :

```
1.<settings>
2....
3.<localRepository>C:\Documents and
  Settings\jm\.m2repository</localRepository>
4....
5.</settings>
```

Installation

Etape 1 :

- Se procurer d'un zip Apache Maven et le dészipper
- Configurer dans la variable d'environnement Path le chemin d'accès au binaire de maven D:\apache-maven-3.8.5-bin\bin;

```
$ mvn -v
Apache Maven 3.8.5
Maven home:
usr/Users/developer/tools/maven/3.8.5
Java version: 17.0.4.1,
```

Etape2 : Création du projet MAVEN

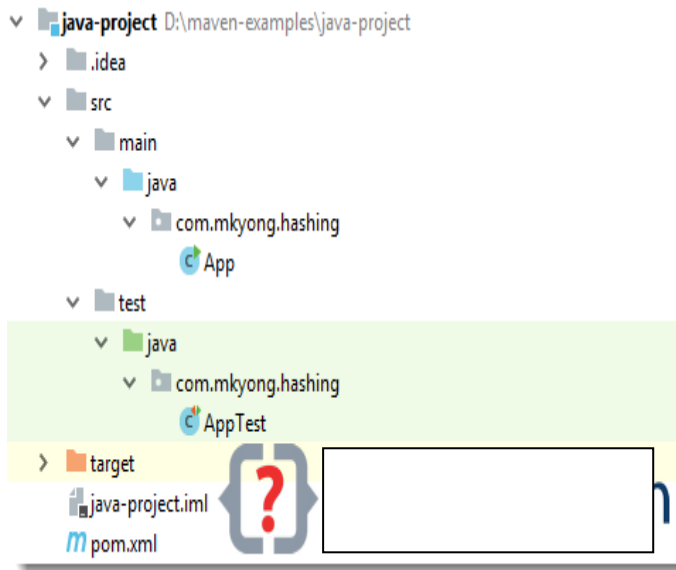
On peut créer un projet avec une template maven avec la commande :

```
$mvn archetype:generate
  -DgroupId={project-packaging}
  -DartifactId={project-name}
  -DarchetypeArtifactId={maven-template}
  -DinteractiveMode=false
```

```
$mkdir projet
$cd projet
$mvn archetype:generate -
DgroupId=com.mkyong.hashing -DartifactId=java-
project -DarchetypeArtifactId=maven-archetype-
quickstart -DinteractiveMode=false
```

NB : choisir maven-archetype-webapp

Arborescence du projet



App.java

```
package com.mkyong.hashing;
```

```
/**
 * Hello world!
 *
 */
public class App
{
    public static void main( String[] args )
    {
        System.out.println( "Hello World!" );
    }
}
```

```
}
```

pom.xml

```
<project
xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"

xsi:schemaLocation="http://maven.apache.org/PO
M/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.mkyong.hashing</groupId>
  <artifactId>java-project</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>java-project</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

```
$ cd java-project
$ mvn install clean (pour télécharger et
ajouter les packages (dépendances jar) au dépôt
local
$ mvn package ou deploy (pour générer un
package de l'application (l'artifact du projet) dans
target/artifactId-version.jar ou war
```

Les dépendances Spring Boot utilisent l' identifiant de groupe org.springframework.boot. Généralement, votre fichier Maven POM hérite du projet spring-boot-starter-parent et déclare des dépendances à un ou plusieurs « [Starters](#) ». Spring Boot fournit également un [plugin Maven](#) en option pour créer des fichiers JAR exécutables.

Structure minimale du fichier pom.xml.

en fournissant plusieurs informations, dont **les starters de dépendances**.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project
xmlns="http://maven.apache.org/POM/4.0.0"
```

```

xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
    xsi:schemaLocation="http://maven.apach
e.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>

    <groupId>org.springframework.boot</gro
upId>

    <artifactId>spring-boot-starter-
parent</artifactId>
    <version>3.2.5</version>
    <relativePath/> <!-- lookup
parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>demo</name>
    <description>Demo project for Spring
Boot</description>
    <properties>

    <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>

        <groupId>org.springframework.boot</gro
upId>

        <artifactId>spring-
boot-starter</artifactId>
        </dependency>

        <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-
boot-starter-test</artifactId>
        <scope>test</scope>
        </dependency>

        <!-- Ajouter des dépendances typiques
pour une application Web -->
        <dependency>
            <groupId>
org.springframework.boot </groupId>
            <artifactId> spring-boot-
starter-web </artifactId>
            </dependency>
        </dependency>

    </dependencies>

```

```

<!-- Package pour pouvoir créer un
fichier jar exécutable -->

<build>
    <plugins>
        <plugin>

        <groupId>org.springframework.boot</gro
upId>

        <artifactId>spring-boot-maven-
plugin</artifactId>
        </plugin>
    </plugins>
</build>

</project>

```

Ecrire le code

Créer un fichier Example.java dans

src/main/java/

```

import org.springframework.boot.* ;
import
org.springframework.boot.autoconfigure.* ;
import org.springframework.stereotype.* ;
import
org.springframework.web.bind.annotation.* ;

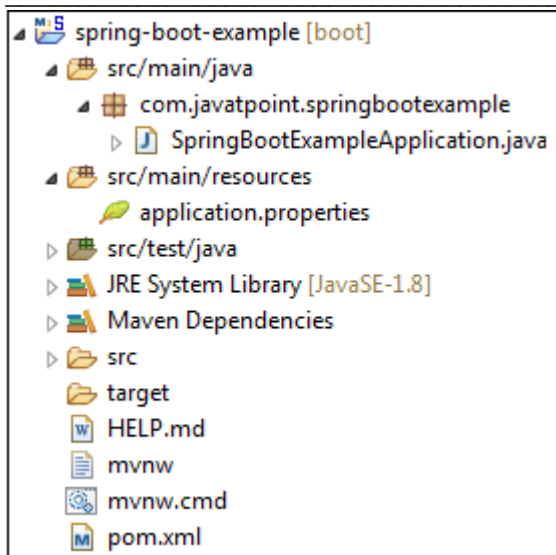
@RestController
@EnableAutoConfiguration
public class Exemple {

    @RequestMapping("/")
    String home() {
        return "Bonjour tout le monde !" ;
    }

    public static void main (String[] args) throws
Exception {
        SpringApplication.run (Exemple.class ,
args);
    }
}

```

Les annotations `@RestController` et `@RequestMapping` sont des annotations Spring MVC. L'annotation `@RestController` indique à Spring de restituer la chaîne résultante directement à l'appelant. L'annotation `@RequestMapping` fournit une information sur le routing.



La structure d'un projet Spring Boot créé par Spring Initializr est la suivante :

- **src/main/java**: contient le fichier d'application principal avec l'annotation **@SpringBootApplication** et toutes les autres classes Java avec les packages que vous ajoutez.
- **src/main/resources**: Contient des fichiers de configuration comme **application.properties** ou **application.yml** et des ressources statiques ou templates (HTML, par exemple).
- **src/test/java**: Contient des classes de test.

SpringBootApplication.java

```
package com.javatpoint.springbootexample;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringBootApplication
{
    public static void main(String[] args)
    {
        SpringApplication.run(SpringBootApplication.class, args);
    }
}
```

L'annotation **@SpringBootApplication** est une simple encapsulation de trois annotations :

1. **@Configuration** : donne à la classe actuelle la possibilité de définir des configurations qui iront remplacer les traditionnels fichiers XML. Ces configurations se font via des Beans.
2. **@EnableAutoConfiguration** : l'annotation vue précédemment qui permet, au

démarrage de Spring, de générer automatiquement les configurations nécessaires en fonction des dépendances situées dans notre classpath.

3. **@ComponentScan** : indique qu'il faut scanner les classes de ce package afin de trouver des Beans de configuration.

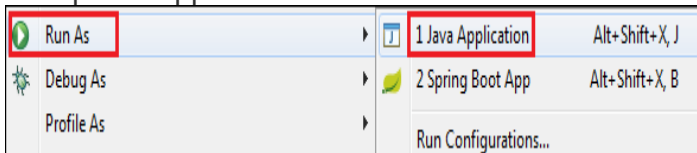
pom.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
3.   <modelVersion>4.0.0</modelVersion>
4.   <parent>
5.     <groupId>org.springframework.boot</groupId>
6.     <artifactId>spring-boot-starter-parent</artifactId>
7.     <version>2.2.2.BUILD-SNAPSHOT</version>
8.     <relativePath/> <!--
        lookup parent from repository -->
9.   </parent>
10.  <groupId>com.javatpoint</groupId>
11.  <artifactId>spring-boot-example</artifactId>
12.  <version>0.0.1-SNAPSHOT</version>
13.  <name>spring-boot-example</name>
14.  <description>Demo project for Spring Boot</description>
15.  <properties>
16.    <java.version>1.8</java.version>
17.  </properties>
18.  <dependencies>
19.    <dependency>
20.      <groupId>org.springframework.boot</groupId>
21.      <artifactId>spring-boot-starter</artifactId>
22.    </dependency>
23.    <dependency>
24.      <groupId>org.springframework.boot</groupId>
25.      <artifactId>spring-boot-starter-test</artifactId>
26.      <scope>test</scope>
27.    </dependency>
28.    <exclusion>
29.      <groupId>org.junit.vintage</groupId>
30.      <artifactId>junit-vintage-engine</artifactId>
31.    </exclusion>
32.  </dependencies>
33.  <build>
34.    <plugins>
35.      <plugin>
36.        <groupId>org.springframework.boot</groupId>
```

```

39. <artifactId>spring-boot-maven-
    plugin</artifactId>
40. </plugin>
41. </plugins>
42. </build>
43. <repositories>
44. <repository>
45. <id>spring-milestones</id>
46. <name>Spring Milestones</name>
47. <url>https://repo.spring.io/milestone</url>
48. </repository>
49. <repository>
50. <id>spring-snapshots</id>
51. <name>Spring Snapshots</name>
52. <url>https://repo.spring.io/snapshot</url>
53. <snapshots>
54. <enabled>true</enabled>
55. </snapshots>
56. </repository>
57. </repositories>
58. <pluginRepositories>
59. <pluginRepository>
60. <id>spring-milestones</id>
61. <name>Spring Milestones</name>
62. <url>https://repo.spring.io/milestone</url>
63. </pluginRepository>
64. <pluginRepository>
65. <id>spring-snapshots</id>
66. <name>Spring Snapshots</name>
67. <url>https://repo.spring.io/snapshot</url>
68. <snapshots>
69. <enabled>true</enabled>
70. </snapshots>
71. </pluginRepository>
72. </pluginRepositories>
73. </project>
    
```

Étape 6 : exécutez le fichier **SpringBootExampleApplication.java**. Faites un clic droit sur le fichier -> Exécuter en tant que -> Applications Java



La figure suivante montre que l'application s'exécute avec succès.

```

: Starting SpringBootExampleApplication on Anubhav-PC with PID 5096 (C:\Users\Anubhav
: No active profile set, falling back to default profiles: default
: Started SpringBootExampleApplication in 41.147 seconds (JVM running for 1856.017)
    
```

On peut exécuter le projet en ligne de commande en configurant dans la variable d'environnement Path le chemin d'accès au binaire de maven D:\apache-maven-3.8.8-bin\bin;

- cd demo
- mvn spring-boot:run // **fait appel à la dépendance spring-boot-starter-parent** pour démarrer l'application

3.2. Création et configuration d'un projet Spring Boot dans l'IDE Eclipse ou à l'aide de STS

Spring Tool Suite (STS) est un IDE basé sur Eclipse pour développer, exécuter, déployer et déboguer l'application Spring.

Installez le plug-in **IDE Spring Tools Suite (STS)** avec Eclipse :

- Ouvrez Éclipse.
- Allez dans **Help-> Eclipse Marketplace**.
- Recherchez « Spring Tools » dans la zone de recherche Marketplace.
- Installez "Spring Tools 4" (ou la dernière version disponible).
- Redémarrez Eclipse après l'installation

On peut aussi installer STS avec l'exécutable **STS.exe** depuis <https://spring.io/tools3/sts/all>

. Voici un guide étape par étape pour Créer et configurer un projet Spring Boot dans Eclipse en particulier avec le plugin Spring Tools Suite (STS):

1. Créez un nouveau projet Spring Starter :
Une fois STS installé, vous pouvez facilement démarrer une application Spring Boot :
Allez dans **File-> New-> Spring Starter Project**.
Dans la boîte de dialogue Nouveau projet Spring Starter :
Nommez votre projet et fournissez d'autres détails tels que le groupe et l'artefact.
Choisissez le package (**jar** c'est typique pour Spring Boot).
Choisissez la version Java que vous souhaitez utiliser.
Cliquez sur **Next**.
Dans la section "Sélectionner les dépendances", choisissez les composants dont vous avez besoin. Pour une application Web de base, sélectionnez "Web" -> **Spring Web**.
Cliquez sur **Finish**.
Eclipse va maintenant générer un nouveau projet Spring Boot avec les dépendances sélectionnées. Ce projet comprendra un fichier d'application principal, le fichier **pom.xml** avec les dépendances et la structure de projet par défaut.

2. Explorez et exécutez l'application :

- Accédez au fichier d'application principal dans le répertoire `src/main/java`. Il aura une annotation `@SpringBootApplication`.
- Faites un clic droit sur le fichier d'application principal.
- Choisissez `Run As -> Java Application` ou `Spring Boot App`.
Votre application Spring Boot va maintenant démarrer et vous verrez des journaux dans la console Eclipse indiquant que le serveur Tomcat intégré est en cours d'exécution.

3. Ajoutez votre code :

Vous pouvez maintenant commencer à créer votre application :

- Créez des contrôleurs, des services et des référentiels selon vos besoins.
 - Mettez à jour le fichier `application.properties` ou `application.yml` dans le répertoire `src/main/resources` pour configurer votre application.
- ### 4. Conseils supplémentaires :
- Vous pouvez utiliser le fichier `pom.xml` pour ajouter des dépendances supplémentaires. Après avoir mis à jour le `pom.xml`, assurez-vous de mettre à jour le projet Maven en cliquant avec le bouton droit sur le projet -> `Maven -> Update Project`.
 - Si vous rencontrez des problèmes avec la configuration du projet, assurez-vous que la bonne version Java JDK est installée et définie dans Eclipse.
Spring Boot fournit une interface nommée `"CommandLineRunner"` pour écrire des résultats dans la console. En implémentant cette interface, la classe sera obligée de déclarer la méthode `"public void run(String... args) throws Exception"`.

Exemple

```
package com.example.helloworld;
```

```
import
org.springframework.boot.CommandLineRunner;
import
org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class HelloworldApplication implements
CommandLineRunner {
```

```
public static void main(String[] args) {
```

```
SpringApplication.run(HelloworldApplication.class,
args);
}
```

```
@Override
```

```
public void run(String... args) throws Exception
{
    System.out.println("Hello World!"); // afficher
le texte "Hello World!" dans la console
}
```

4. Inversion de contrôle et injection des dépendances

L'inversion de contrôle (IoC) est un principe de conception dans lequel le contrôle de la création d'objets et de son cycle de vie est transféré de la routine principale de l'application vers un conteneur ou un framework externe.

L'injection de dépendances est une technique dans laquelle un objet fournit les dépendances d'un autre objet, plutôt que l'objet dépendant les créant lui-même manuellement. En d'autres termes, il s'agit d'un moyen d'obtenir une inversion de contrôle (IoC), dans laquelle le contrôle du flux d'exécution d'un programme est transféré du programme lui-même vers un framework ou un conteneur.

L'injection de dépendances (DI) est une méthode populaire pour réaliser l'IoC, où les objets reçoivent leurs dépendances d'une source externe plutôt que de les créer en interne.

L'injection de dépendances est une implémentation spécifique d'IoC permettant à une application de déléguer la gestion du cycle de vie de ses dépendances et leurs injections à une autre entité.

Exemple 1 de l'Inversion de contrôle

L'inversion de contrôle (IoC) est un principe de conception dans lequel le flux de contrôle d'un programme est inversé. Au lieu que le développeur contrôle le flux, un framework ou un conteneur prend le contrôle

```
// Exemple sans IoC
public class WithoutIoC {
    public static void main ( String [] args )
    { Service service = new Service (); // difficile de
      réutiliser le code pour contenir une autre service
        service.faire_quelque_chose (); } }

// Exemple avec IoC avec injection de
dépendances
public class WithIoC {
    private Service service ;
    // Constructeur pour Injection de n'importe que
service
    public WithIoC ( Service service ) { this.service
= service ; }
    public void executer () {
        service.faire_quelque_chose (); }
}
```

Exemple 2 d'inversion de controle.

Imaginez un scénario classique dans lequel vous disposez d'une application qui envoie des notifications.

1. Sans IoC, vous pourriez avoir :

```
public class EmailService {
    public void sendEmail ( String message , String
Receiver ){ // logique d'envoi d'e-mail }
}

class public Notification {
    private EmailService emailService = new
EmailService ();
    public void notifyUser (String message , String
Receiver) {
        emailService.sendEmail ( message ,
Receiver ); } }
```

Dans l'exemple ci-dessus, la classe **Notification** instancie directement **EmailService**. Il sera donc difficile de modifier le mécanisme de notification à l'avenir (par exemple, passer aux SMS ou à une application de messagerie).

2. Maintenant, **implémentons cela avec IoC** à l'aide de l'injection des dépendances (DI) :

```
public interface MessageService {
    void sendMessage (String msg , String Receiver );
}

public class EmailServiceImpl implements
MessageService {
    public void sendMessage ( String msg , String
Receiver ){
```

```
// logique d'envoi d'e-mail
}
}

public class SMSServiceImpl implements
MessageService {
    public void sendMessage ( String msg , String
Receiver ){
// logique pour envoyer des SMS
}
}

public class Notification {
    private MessageService service;

    public Notification ( MessageService svc ){
this.service = svc ; }
    public void notifyUser (String message , String
Receiver) {
        service.sendMessage ( message , Receiver
); } }
```

Dans ce code refactorisé :

1. Nous avons défini une interface **MessageService**.
2. Nous avons deux implémentations de **MessageService** : **EmailServiceImpl**(envoi des e-mails) et **SMSServiceImpl**(envoi des SMS).
3. La classe **Notification** prend désormais un **MessageService** comme argument du constructeur (injection de dépendance). Lorsque vous souhaitez envoyer une notification par e-mail :

```
MessageService emailService = new
EmailServiceImpl ();
Notification notification = new Notification (
emailService );
notification.notifyUser ( "Bonjour" ,
"utilisateur@exemple.com" );
```

Ou si vous souhaitez envoyer une notification par SMS :

```
MessageService smsService = new
SMSServiceImpl ();
Notification notification = new Notification (
smsService );
notification.notifyUser ( "Bonjour" , "1234567890"
);
```

Avec IoC en place, la classe **Notification** n'a pas besoin d'être modifiée si vous introduisez une nouvelle méthode d'envoi de messages. Tout ce dont vous avez besoin est une nouvelle implémentation de **MessageService**.

Dans les applications du monde réel, des frameworks comme Spring ou Java EE fournissent le conteneur IoC, et vous n'instanciez pas manuellement les classes ni n'injectez de dépendances comme dans l'exemple ci-dessus. Au lieu de cela, le conteneur IoC gère la création d'objets, le cycle de vie et l'injection de dépendances.

Dans le contexte de Spring, l'injection de dépendances (DI) est utilisée pour gérer les dépendances entre différents composants d'une application, tels que les classes, les beans ou les services. Spring fournit un conteneur (souvent appelé « conteneur Spring IoC ») chargé d'instancier, de configurer et de gérer ces objets. Le conteneur injecte les dépendances requises dans les objets au moment de l'exécution, vous permettant de créer des applications faiblement couplées et hautement maintenables.

Comment fonctionne l'injection de dépendances en Spring ?

- **Composants** : dans une application Spring, vous définissez généralement divers composants, tels que des classes ou des beans, qui représentent différentes parties des fonctionnalités de votre application.
- **Dépendances** : ces composants ont souvent des dépendances vis-à-vis d'autres composants ou services pour effectuer leurs tâches efficacement.
- **Configuration** : vous configurez votre application Spring à l'aide de fichiers de configuration basés sur XML, d'annotations Java ou de classes de configuration basées sur Java (à l'aide des annotations `@Configuration` et `@Bean`).
- **Injection** : le conteneur IoC de Spring gère la création des objets et de leurs dépendances. Lorsqu'un composant a besoin d'une dépendance, le conteneur l'injecte au moment de l'exécution, garantissant que les objets requis sont disponibles et correctement initialisés.

Il existe deux manières principales d'effectuer l'injection de dépendances en Spring Boot :

- **Injection de constructeur** : les dépendances sont injectées via le constructeur de la classe dépendante. Il s'agit de l'approche la plus courante et la plus recommandée
- **Injection Setter** : les dépendances sont injectées via des méthodes setter dans la classe dépendante.

// Java Program to Illustrate Principal Class

```
package BeanAnnotation;
public class Principal {

    // Method
    public void principalInfo()
    {
        // Print statement
        System.out.println("Salut, je suis votre
directeur");
    }
}
```

Voie 1 : Injection de dépendances via un constructeur (CDI)

A. Classe collégiale

```
// Java Program to Illustrate College Class

package BeanAnnotation;

// Class
public class College {

    private Principal principal;

    public College(Principal principal)
    {
        this.principal = principal;
    }

    public void test()
    {
        principal.principalInfo();
        System.out.println("Test College Method");
    }
}
```

B. Classe de configuration avec Injection de dépendances avec l'annotation @Bean

```
// Java Program to Illustrate Configuration Class

package BeanAnnotation;

// Importing required classes
import org.springframework.context.annotation.Be
import org.springframework.context.annotation.Co

// Annotation
@Configuration
public class CollegeConfig {

    // Creating the Bean for Principal Class
```

```
@Bean
public Principal principalBean()
{
    return new Principal();
}

@Bean
public College collegeBean()
{
    // Constructor Injection
    return new College(principalBean());
}
```

Voie 2 : Injection de dépendance de setter (SDI)

A. Classe collégiale

// Java Program to Illustrate College Class

```
package BeanAnnotation;

// Class
public class College {

    // Class data members
    private Principal principal;
    // Setter
    public void setPrincipal(Principal principal)
    {

        // this keywords refers to current instance
        itself
        this.principal = principal;
    }

    // Method
    public void test()
    {
        principal.principalInfo();

        // Print statement
        System.out.println("Test College Method");
    }
}
```

B. Classe de configuration

// Java Program to Illustrate Configuration Class

```
package BeanAnnotation;

// Importing required classes
import
org.springframework.context.annotation.Bean;
import
org.springframework.context.annotation.Configuration;

// Annotation
@Configuration
public class CollegeConfig {

    // Creating the Bean for Principal Class
    @Bean
    public Principal principalBean()
    {

        return new Principal();
    }
}
```

C. Classe de candidature Main.java

// Java Program to Illustrate Application Class

```
package BeanAnnotation;
// Importing required classes
import
org.springframework.context.ApplicationContext;
import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

// Main class
public class Main {

    // Main driver method
    public static void main(String[] args)
    {
        // Using AnnotationConfigApplicationContext
        // instead of ClassPathXmlApplicationContext
        // because we are not using XML
        Configuration
        ApplicationContext context
        = new
        AnnotationConfigApplicationContext(
        CollegeConfig.class);

        // Getting the bean
        College college
        = context.getBean("collegeBean",
        College.class);

        // Invoking the method
        // inside main() method
        college.test();
    }
}
```

Sortie:

Salut, je suis votre directeur

Test College Method

```

@Bean
public College collegeBean()
{

    // Setter Injection
    College college = new College();
    college.setPrincipal(principalBean());

    return college;
}
}

```

C. Classe de candidature Main.java

```

// Java Program to Illustrate Application (Main)
Class

package BeanAnnotation;

// Importing required classes
import
org.springframework.context.ApplicationContext;
import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

// Application (Main) class
public class Main {

    // Main driver method
    public static void main(String[] args)
    {

        // Using AnnotationConfigApplicationContext
        // instead of ClassPathXmlApplicationContext
        // because we are not using XML
        Configuration
        ApplicationContext context = new
        AnnotationConfigApplicationContext(
            CollegeConfig.class);

        // Getting the bean
        College college =
        context.getBean("collegeBean", College.class);

        // Invoking the method
        // inside main() method
        college.test();
    }
}

```

Sortie:

Salut, je suis votre directeur

Test College Method

5. Création des beans avec l'annotation @Component et injection de dépendances avec @Autowired

Spring Framework fournit l'IoC container qui va instancier les classes, puis si nécessaire les injecter dans d'autres instances de classe. Un bean est une classe au sein du contexte Spring (l'IoC container). Pour qu'une classe soit manipulée par l'IoC container, il est nécessaire de l'indiquer explicitement à Spring de 2 manières : La première solution est l'utilisation de fichiers XML au sein desquels on décrit les classes que Spring doit gérer ; voici un exemple simple :

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD
BEAN 2.0//EN"
"http://www.springframework.org/dtd/spring-
beans-2.0.dtd">

```

```

<beans>
    <bean id="myBean"
class="com.exa.BeanImpl"/>
</beans>

```

La seconde solution est l'utilisation des annotations.

Voici quelques exemples d'annotations Spring :

- @Component
- @Autowired
- @Qualifier

Par exemple ,
package com.exa;

```

import
org.springframework.beans.factory.annotation.Qualifier;
import
org.springframework.stereotype.Component;

```

```

@Component
@Qualifier("myBean")
public class BeanImpl {
    // TO DO
}

```

L'annotation **@Component** permet de déclarer la classe BeanImpl auprès de Spring, comme étant un bean à avoir dans l'IoC container.

L'annotation **@Qualifier** permet de donner un nom à ce bean, en l'occurrence "myBean" (si cette annotation n'est pas définie, le nom par défaut est le nom de la classe).

L'annotation **@Autowired** : injecte des @Component dans une classe pour les utiliser.


```
public class MailConfig {
    @Autowired
    private ApplicationProperties
    applicationProperties;
}
```

L'annotation **@Service** : classe qui réalise des traitements métiers sur des données.

```
@Service
public class StorageService {
}
```

L'annotation **@Repository** : classe qui manipule des données d'une BDD cad les classes DAO chargées de fournir des opérations CRUD sur les tables de base de données.

```
@Repository
public interface UserRepository extends
JpaRepository<User, Long> {
}
```

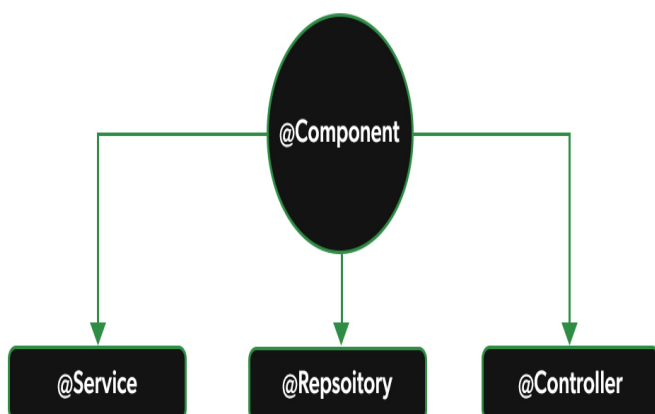
L'annotation **@Controller** / **@RestController** : classe exposant une API REST.

```
@Controller
public class UserController {
}
```

@RequestMapping : URI de base d'accès à votre API REST (« /api »)

```
@RestController
@RequestMapping("/api")
public class UserResource {
}
```

Résumé des annotations **@Component**, **@Service**, **@Controller** et **@Repository** dans le framework Spring :



• **@Service**, **@Repository** ou **@Controller**, qui sont des spécialisations de **@Component**

@Component est une annotation que vous pouvez utiliser pour marquer une classe en tant que composant qui doit être analysé et enregistré en tant que bean dans le contexte de l'application. On peut instancier et injecter ce bean dans d'autres beans avec l'annotation **@Autowired**.

Exemple 1 de création d'un bean

```
package com.example.demo;
import
org.springframework.stereotype.Component;
import org.springframework.stereotype.Service;
import
org.springframework.stereotype.Repository;
```

@Component

```
//@Repository
//@Service
//@Component("monbean") // donner un nom
public class MonBean {
    public void afficher()
    {
        System.out.println( "Bonjour KOTO
");
    }
    public int factorial(int n)
    {
        if (n == 0)
            return 1;
        else
            return n*factorial(n - 1);
    }
}
```

Exemple 2 : créer un bean via une classe de service

```
// créer une simple classe
package com.example.demo;
public class HelloWorld {
    String value ="Hellod World";
    @Override
    public String toString() {
        return "VALUE" + value;
    }
}
```

```
// créer la classe BusinessService.java
déclarée en tant que bean
// dans le context Spring. Via une
annotation @Component
```

```
package com.example.demo;
import
org.springframework.stereotype.Component;
@Component
public class BusinessService {
    public HelloWorld getHelloWorld() {
        return new HelloWorld();
    }
}
```

Exemple 3 : créer un bean en tant qu'une implémentation d'une interface

```
package com.example.demo;
public interface MonInterface {
    public void afficher();}
// classe d'implémentation
package com.example.demo;
import
org.springframework.stereotype.Component;
@Component
public class HelloWorldImpl implements
MonInterface {
    @Override
    public void afficher()
    {System.out.println( "BONSOIR RIVO");
    }
}
// classe principale
package com.example.demo;
import
org.springframework.boot.CommandLineRunner;
import
org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.beans.factory.annotation.Autowired;
```

```
@SpringBootApplication
public class HelloworldApplication implements
CommandLineRunner {
    @Autowired
    private MonBean monbean;
    @Autowired
    private BusinessService bs;
    @Autowired
    private HelloWorldImpl helloImpl;
    public static void main(String[] args) {

SpringApplication.run(HelloworldApplication.class,
args);
    }
}
```

```
@Override
public void run(String... args) throws Exception
{
    monbean.afficher();
    int factorial5 = monbean.factorial(5);
    System.out.println("Factorial of 5 is: "+ factorial5);
    HelloWorld hw = bs.getHelloWorld();
    System.out.println(hw);
    helloImpl.afficher();
}
}
```

C'est grâce à l'IoC container de Spring , on applique l'**injection de dépendances**. En mettant l'annotation **@Autowired** sur l'attribut bs, **Spring va chercher au sein de son contexte s'il existe un bean de type BusinessService**.

✓ S'il le trouve, il va alors instancier la classe de ce bean et **injecter cette instance dans l'attribut**.

✗ S'il ne trouve pas de bean de ce type, Spring génère une erreur.

En résumé

- Au sein d'une application Spring Boot, écrire du code implique de **définir les beans** utilisés par Spring.
- On peut exécuter du code grâce à l'implémentation de l'interface **CommandLineRunner** et de la méthode **run**.
- Pour qu'une classe soit **déclarée en tant que bean**, on l'annoté **@Component**.
- Pour qu'un bean **soit injecté** dans un attribut, on annoté l'attribut **@Autowired**
- Tous les composants annotés **@Component** , **@Service** et **@Repository** sont enregistrés dans un contexte d'application **ApplicationContext** et sont accessibles à l'aide de la classe **AnnotationConfigApplicationContext**. Par exemple,

```
AnnotationConfigApplicationContext context =new
AnnotationConfigApplicationContext(HelloworldAp
plication.class);
```

```
MonBean mb =
context.getBean(MonBean.class);
mb.afficher();
```

ou bien

```
AnnotationConfigApplicationContext context =new
AnnotationConfigApplicationContext();
context.scan("com.example.demo");
context.refresh();
```

```
MonBean mb =
context.getBean(MonBean.class);
mb.afficher();
```

6. Création des beans avec l'annotation @Bean

La principale différence entre les annotations @Component et @Bean est que @Component est utilisé pour marquer une classe en tant que bean tandis que @Bean est utilisé pour déclarer une méthode qui retourne **un objet qui doit être inscrit en tant que bean dans le contexte de l'application** et généralement utilisé dans les classes de configuration annotée avec @Configuration,

Le nom de la méthode est généralement le nom par défaut du bean, mais vous pouvez également spécifier un nom différent à l'aide de l'attribut value de l'annotation. Par exemple, vous pouvez créer une méthode @Bean qui retourne un objet DataSource pour l'accès à la base de données

@Configuration

```
public class AppConfig {
```

@Bean

```
    public DataSource dataSource() {
//créer et renvoyer un objet DataSource
        DriverManagerDataSource dataSource =
new DriverManagerDataSource();
```

```
        dataSource.setDriverClassName("com.mysql.jdbc
.Driver");
```

```
        dataSource.setUrl("jdbc:mysql://localhost:3306/ba
se");
        dataSource.setUsername("root");
        dataSource.setPassword("root");
        return dataSource;
    }
}
```

7. Injection de dépendances avec @Autowired et @Qualifier

@Autowired est utilisé pour injecter automatiquement des dépendances dans une classe. Lorsque vous annotez un champ, une méthode de définition ou un constructeur avec @Autowired, Spring tentera de trouver un bean correspondant dans le contexte de l'application et de l'injecter dans le composant annoté. Il est généralement utilisé lorsque vous avez plusieurs beans du même type et que Spring doit déterminer lequel injecter automatiquement.

L'annotation **@Qualifier** est utilisé conjointement avec **@Autowired** pour spécifier quel bean exact

doit être injecté lorsqu'il existe plusieurs beans du même type dans le contexte de l'application. Vous utilisez @Qualifier pour fournir un nom ou une valeur de bean spécifique pour indiquer quel bean doit être injecté. .

Exemple 1

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;
```

@Service

```
public class MyService {
    private final MyRepository firstRepository;
    private final MyRepository secondRepository;
```

@Autowired

```
    public MyService(@Qualifier("firstRepository")
        MyRepository firstRepository,
        @Qualifier("secondRepository")
        MyRepository secondRepository)
```

```
    {
        this.firstRepository = firstRepository;
        this.secondRepository = secondRepository;
    }
}
```

Ce code Java définit une classe de service Spring nommée MyService avec deux dépendances de type MyRepository. Le constructeur @Autowired utilise l'injection de dépendances, et les annotations @Qualifier sont utilisées pour spécifier à injecter pour chaque dépendance. Ce code illustre le mécanisme d'injection de dépendances de Spring pour les composants d'une application. Ici, nous utilisons @Qualifier pour injecter les beans MyRepository.

Exemple 2

Classe Students

```
public class Students {
    // Attributes
    private String name;
    private int age;
    // Generating the constructor
    public Students(String name, int age)
    {
        this.name = name;
        this.age = age;
    }

    // Getter-setters
    public String getName() { return name; }
    public void setName(String name) { this.name
= name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}
```

Classe Profile dans laquelle on injecte la dépendance Student2

```
package com.example.demo;
import
org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;
//@Component
public class Profile {
    @Autowired
    @Qualifier("student2")
    private Students student;
    public Profile(){
        System.out.println("Inside Profile constructor.");
    }
    public void printAge() {
        System.out.println("Age : " + student.getAge());
    }
    public void printName() {
        System.out.println("Name : " + student.getName());
    }
}
```

AppConfig.java

```
package com.example.demo;
import org.springframework.context.annotation.Bean;
import
org.springframework.context.annotation.Configuration;
```

```
@Configuration
public class AppConfig {
```

```
    @Bean
    public Profile profileBean()
    {
        // Returns an object
        return new Profile();
    }
```

```
@Bean(name = "student1")
public Students studentBean1()
{
    // Returns an object
    return new Students("KOTO",20);
}
```

```
    @Bean(name = "student2")
    public Students studentBean2()
    {
        // Returns an object
        return new Students("RIVO",25);
    }
```

```
}

// programme principal
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.context.ApplicationContext;
import
org.springframework.context.annotation.AnnotationConfig
ApplicationContext;
import org.springframework.context.annotation.Bean;
```

```
@SpringBootApplication
public class HelloworldApplication implements
CommandLineRunner {

    public static void main(String[] args) {
        SpringApplication.run(HelloworldApplication.class,
args);
    }
```

```
    @Override
    public void run(String... args) throws Exception {
```

```
        AnnotationConfigApplicationContext context =new
AnnotationConfigApplicationContext(AppConfig.class);
```

```
        Profile profile = (Profile)
context.getBean("profileBean","Profile.class");
        profile.printName();// RIVO
        profile.printAge();// 25
    }
```

```
}
```

8.Accès aux bases des données en Spring Boot

La classe JdbcTemplate du framework Spring Boot fournit plusieurs méthodes surchargées pour exécuter différents types de requêtes SQL et effectuer des opérations CRUD.

Dépendances nécessaires pour utiliser JdbcTemplate avec SpringBoot

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

Mappage des résultats d'une requête SQL

Avec Spring JdbcTemplate, on peut récupérer des résultats d'une requête SQL sous forme d'une liste d'objets à la manière d'un ORM.

On peut notamment utiliser BeanPropertyRowMapper, une implémentation de RowMapper fournie par Spring permettant de mapper automatiquement les lignes retournées vers nos entités.

I. BeanPropertyRowMapper

Ce mapper est très pratique à utiliser car il permet de récupérer une liste d'objets simples sans aucune configuration. On va voir dans cette section comment il marche et ses limites:

```
public class EtudiantEntity {

    private Long id;
    private String name;
    public void setId(Long id) {
        this.id = id;
    }

    public Long getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

Pour récupérer une liste des étudiants, on peut utiliser la méthode query() de JdbcTemplate qui prend en paramètre une requête SQL et un RowMapper.

```
@Override
public Collection<EtudiantEntity> findAll() {
    return jdbcTemplate.query("SELECT id, name
FROM Etudiant", new
BeanPropertyRowMapper<>(EtudiantEntity.class)
);
}
```

On remarque que BeanPropertyRowMapper prend en paramètre la classe vers laquelle transformer les résultats.

Pour effectuer le mapping, il va prendre les résultats de la requête et appeler les setters correspondants.

Le mapper va déterminer les setters à utiliser suivant le nom des colonnes

id -> setId

name -> setName

II. Utiliser un RowMapper personnalisé

Imaginons que l'objet Etudiant défini précédemment ait en attribut un objet quelconque. Dans la table, on a une colonne "objectId" avec l'id d'un objet. On devrait alors faire un Mapper:

```
public class EtudiantEntityRowMapper
implements RowMapper<EtudiantEntity> {

    @Override
    public EtudiantEntity mapRow(ResultSet rs, int
rowNum) throws SQLException {
        EtudiantEntity entity = new EtudiantEntity();
        entity.setId(rs.getLong("id"));
        entity.setName(rs.getString("name"));

        entity.customMethod(rs.getString("objectId"));
        return entity;
    }
}
```

Puis l'appeler via

```
@Override
public Collection<EtudiantEntity> findAll() {
    return jdbcTemplate.query("SELECT id, label as
name FROM Etudiant", new
EtudiantEntityRowMapper());
}
```

Exemples JdbcTemplate dans Spring Framework pour effectuer des requêtes

Voici à quoi ressemblent notre base de données et nos tables :

```
mysql> SELECT * FROM employe ;
+-----+-----+-----+
| emp_id | nom_emp | identifiant_départ | salaire |
+-----+-----+-----+
| 101 | Jacques | 1 | 1000 |
| 102 | Kate | 1 | 1200 |
| 103 | Jacques | 2 | 1400 |
| 104 | Jean | 2 | 1450 |
| 105 | Johnny | 3 | 1050 |
| 108 | Alain | 3 | 1150 |
| 106 | Virat | 4 | 850 |
| 107 | Vina | 4 | 700 |
```



```
| 109 | joie | 4 | 700 |
| 110 | Jacques | 1 | 1000 |
+-----+-----+-----+-----+
10 lignes dans un ensemble (0,00 sec)
```

```
mysql> SHOW TABLES ;
```

```
+-----+
| Tables_in_test |
+-----+
| livres          |
| département     |
| employe         |
+-----+
```

```
3 lignes par série (0,09 sec)
```

Et voici quelques exemples courants d'interaction avec la base de données et d'exécution d'une requête SQL pour lire et écrire des données à partir de tables à l'aide de la classe `JdbcTemplate` du framework Spring.

1. Comment utiliser `JdbcTemplate` pour interroger une valeur unique telle que le nombre, l'identifiant, etc.

Si vous souhaitez exécuter une requête SQL qui exécute des fonctions d'agrégation telles que `count()`, `avg()`, `max()` et `min()` ou simplement renvoyer une valeur entière, vous pouvez utiliser la méthode `queryForInt()` de `JdbcTemplate` pour exécuter la requête SQL comme indiqué dans l'exemple suivant :

```
int total = jdbcTemplate.queryForInt("SELECT
count() FROM employee");
logger.info("Total Employees : " + total);
```

2. Exemple `JdbcTemplate` pour interroger et remplir un objet Java à partir de la base de données

Si vous souhaitez exécuter une requête SQL qui renvoie un objet de valeur tel que `String`, vous pouvez utiliser la méthode `queryForObject()` de la classe `JdbcTemplate`. Cette méthode prend un argument sur le type de requête de classe qui sera renvoyée, puis convertit le résultat en cet objet et le renvoie à l'appelant.

```
String name =
jdbcTemplate.queryForObject("SELECT
emp_name FROM employee where emp_id=?",
    new Object[]{103}, String.class);
```

3. Exemple `JdbcTemplate` pour récupérer un objet personnalisé de la base de données

Si votre requête SQL doit renvoyer un objet utilisateur tel que `Employee`, `Order` ou tout autre élément spécifique au domaine, vous devez fournir une implémentation `RowMapper` à la méthode `queryForObject()`. Ce mappeur indiquera à `JdbcTemplate` comment convertir le `ResultSet` en un objet personnalisé.

Voici un exemple de récupération d'un objet personnalisé.

```
Employee emp =
jdbcTemplate.queryForObject("SELECT FROM
employee
                                where emp_id=?",
                                new Object[]{103},
                                new
EmployeeMapper());
```

4. Exemple `JdbcTemplate` pour récupérer une liste d'objets de la table

Si votre requête SQL doit renvoyer une liste d'objets au lieu d'un seul objet, vous devez utiliser la méthode `query()` de `JdbcTemplate`. C'est l'une des méthodes les plus génériques et elle peut exécuter tout type de requête.

Encore une fois, pour convertir le résultat en objet personnalisé, vous devez fournir une implémentation `RowMapper` comme indiqué dans l'exemple suivant :

```
List<Employee> empList =
jdbcTemplate.query("SELECT FROM employee
where salary > 500",
    new EmployeeMapper());
```

5. Comment insérer des enregistrements dans SQL à l'aide de l'exemple Spring `JdbcTemplate`

Vous allez maintenant voir comment écrire des données dans une table, par exemple en exécutant une requête d'insertion, de mise à jour et de suppression à l'aide de `JdbcTemplate`.

Pour insérer des données dans une base de données, vous pouvez utiliser la méthode `update()` de la classe `JdbcTemplate` comme indiqué ci-dessous :

```
int insertCount = jdbcTemplate.update("INSERT
INTO employee values (?, ?, ?, ?)", "111",
"Peter", "1", "2000");
```

6. Comment METTRE À JOUR les enregistrements dans SQL à l'aide de l'exemple Spring JdbcTemplate

La même méthode de mise à jour que nous avons utilisée pour insérer des données dans l'exemple précédent peut également être utilisée pour exécuter la requête de mise à jour dans l'application Spring JDBC.

Voici un exemple de mise à jour d'un enregistrement particulier à l'aide de la classe JdbcTemplate de Spring :

```
int updateCount =
jdbcTemplate.update("UPDATE employee
SET dept_id=? WHERE emp_id=?", "2", "112");
```

7. Comment SUPPRIMER des lignes dans une table à l'aide de Spring JdbcTemplate

La même méthode de mise à jour utilisée pour exécuter la requête d'insertion et de mise à jour peut également être utilisée pour exécuter la requête de suppression, comme indiqué ci-dessous.

Cette fois, il renvoie le nombre de lignes supprimées par une requête SQL donnée, contrairement au nombre d'enregistrements insérés et mis à jour dans les exemples précédents.

```
int deleteCount = jdbcTemplate.update("DELETE
FROM employee WHERE dept_id=?", "1");
```

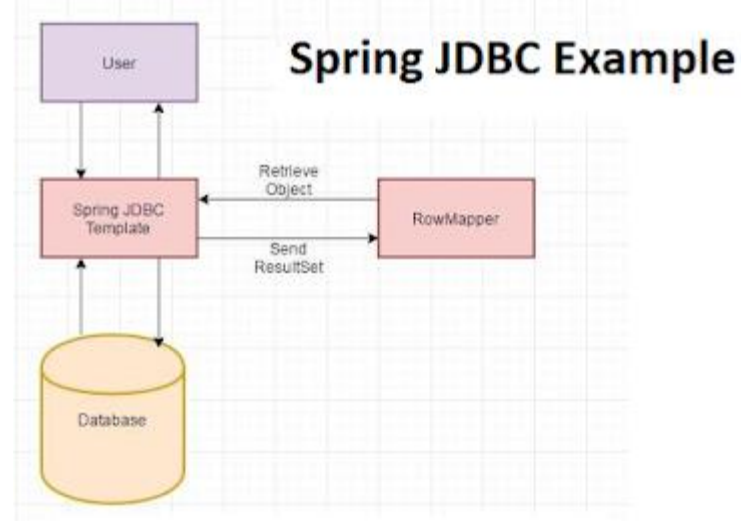
8. Exemple JdbcTemplate pour exécuter n'importe quelle requête SQL

JdbcTemplate peut également exécuter des requêtes DDL telles que Create table ou Create Index.

La classe JdbcTemplate dispose également d'une méthode générique execute() pour exécuter des requêtes DDL comme indiqué ci-dessous où nous avons créé une nouvelle table appelée Book :

```
jdbcTemplate.execute("create table Books (id
integer, name varchar(50), ISBN integer);
```

Exemple de JdbcTemplate du Spring Framework en Java



Voici l'exemple de programme qui vous apprendra à utiliser JdbcTemplate dans une application Java basée sur Spring.

```
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.List;
import javax.sql.DataSource;
import org.apache.log4j.Logger;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import com.test.domain.Employee;
```

```
public class JdbcUtil {
```

```
    private static Logger logger =
    Logger.getLogger(JdbcUtil.class);
```

```
    private JdbcTemplate jdbcTemplate;
```

```
    public void setDataSource(DataSource source){
        this.jdbcTemplate = new
        JdbcTemplate(source);
    }
```

```
    /**
     * This method demonstrates 10
     * JdbcTemplate examples in Spring
     */
```

```

public void jdbcTemplateExamples(){

    // how to use JdbcTemplate to query for
    single value e.g. count, id etc
    // comment utiliser JdbcTemplate pour
    interroger une valeur unique, par exemple
    nombre, identifiant, etc.

    int total =
    jdbcTemplate.queryForInt("SELECT count()
    FROM employee");
    logger.info("Total Employees : " + total);

    //another example to query for single
    value using bind variable in Java
    String name = jdbcTemplate.queryForObject(
    "SELECT emp_name FROM employee where
    emp_id=?", new Object[]{103}, String.class);
    logger.info("Name of Employee : " + name);

    //JdbcTemplate example to query and
    populate Java object from database
    Employee emp =
    jdbcTemplate.queryForObject("SELECT FROM
    employee where emp_id=?", new Object[]{103},
    new EmployeeMapper());
    logger.info(emp);

    //JdbcTemplate example to retrieve a list
    of object from database
    List empList = jdbcTemplate.query("SELECT
    FROM employee where salary > 500",
    new EmployeeMapper());

    logger.info("size : " + empList.size() + ", List of
    Employees : " + empList);

    // JdbcTemplate Example to INSERT
    records into database
    int insertCount =
    jdbcTemplate.update("INSERT INTO employee
    values (?, ?, ?, ?)", "111", "Peter", "1", "2000");
    logger.info("number of rows inserted using
    JdbcTemplate : " + insertCount);

    // How to update records in SQL using
    Spring JdbcTemplate example
    int updateCount =
    jdbcTemplate.update("UPDATE employee
    SET dept_id=? where emp_id=?", "2", "112");

```

```

    logger.info("number of rows updated with
    JdbcTemplate : " + updateCount);

    // How to delete rows in a table using
    Spring JdbcTemplate
    int deleteCount =
    jdbcTemplate.update("DELETE FROM employee
    where dept_id=?",
    "1" );

    logger.info("number of rows deleted
    using JdbcTemplate : "
    + deleteCount);

    // JdbcTemplate example to execute
    any SQL query
    jdbcTemplate.execute("create table
    Books (id integer, name varchar(50), ISBN
    integer)");

    }

    public static void main(String args[]){
    ApplicationContext context
    = new ClassPathXmlApplicationContext("spring-
    config.xml");
    JdbcUtil jdbcUtil = (JdbcUtil)
    context.getBean("jdbcUtil");

    //calling jdbcTemplateExmaples() to
    // demonstrate various ways to use
    JdbcTemplate in Spring
    jdbcUtil.jdbcTemplateExamples();

    }

    /**
     * nested static class to act as
     RowMapper for Employee object
     */

    private static class EmployeeMapper
    implements RowMapper {

        public Employee mapRow(ResultSet rs,
        int rowNum) throws SQLException {

            Employee emp = new Employee();
            emp.setId(rs.getInt("emp_id"));

```

```

emp.setName(rs.getString("emp_name"));

emp.setDepartmentId(rs.getInt("dept_id"));
emp.setSalary(rs.getInt("salary"));
return emp;

    }

}

import
org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.boot.CommandLineRunner;
import
org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.jdbc.core.JdbcTemplate;

import
org.springframework.jdbc.core.RowMapper;
import java.sql.ResultSet;
import java.sql.SQLException;

import java.util.Arrays;
import java.util.List;

@SpringBootApplication
public class DemoApplication implements
CommandLineRunner {

    @Autowired
    JdbcTemplate jdbcTemplate;

    public static void main(String[] args) {

SpringApplication.run(DemoApplication.class,
args);
    }

    @Override
    public void run(String... args) throws Exception
,NullPointerException{
        System.out.println ("StartApplication...");
        runJDBC();
    }

```

```

void runJDBC() {

    System.out.println("Creating tables");
    jdbcTemplate.execute("drop table if exists
customers");
    jdbcTemplate.execute("create table
customers(id int, first_name varchar(255),
last_name varchar(255))");
    // comment utiliser JdbcTemplate pour
interroger une valeur unique, par exemple
nombre, identifiant, etc.

        // int total = (int)
jdbcTemplate.queryForInt("SELECT count()
FROM employee");
        int total = (int)
jdbcTemplate.queryForObject("SELECT count(*)
FROM employee", Integer.class);

    System.out.println ("Total Employees : " + total);
    //un autre exemple pour
interroger une valeur unique à l'aide d'une
variable de liaison dans Java

        String name =
jdbcTemplate.queryForObject("SELECT name
FROM employee where id=?", new Object[]{10},
String.class);
    System.out.println ("Name Employee : " + name);

    // Exemple JdbcTemplate pour interroger
et remplir un objet Java à partir de la base de
données

        Employee emp = (Employee)
jdbcTemplate.queryForObject("SELECT * FROM
employee where id=?", new Object[]{10}, new
EmployeeMapper());

    System.out.println ("Object Employees : " + emp);
    // Exemple JdbcTemplate pour
récupérer une liste d'objets de la base de
données

        List<Employee> empList =
jdbcTemplate.query("SELECT * FROM employee
where salary > 1000", new EmployeeMapper());

    System.out.println ("List Employees : "+empList);
    // Exemple JdbcTemplate pour
INSÉRER des enregistrements dans la base de
données

```

```
int insertCount = jdbcTemplate.update("INSERT
INTO employee values (?, ?, ?)", 57, "Peter", 5555
);
```

```
System.out.println ("nombre de lignes
insérées : " + insertCount);
```

```
int updateCount =
jdbcTemplate.update("UPDATE employee SET
salary=? WHERE id=?", 7777, 11);
```

```
System.out.println ("nombre de lignes
modifiées : " + updateCount);
int deleteCount = jdbcTemplate.update("DELETE
FROM employee WHERE id=?", 13 );
```

```
System.out.println ("nombre de lignes
supprimées : " + deleteCount);
}
```

//Classe statique interne pour agir en tant qu'objet RowMapper pour Employee

```
private static class EmployeeMapper
implements RowMapper {
```

```
public Employee mapRow(ResultSet rs, int
rowNum) throws SQLException {
```

```
Employee emp = new Employee();
emp.setId(rs.getInt("id"));
emp.setName(rs.getString("name"));
```

```
Employee
emp.setSalary(rs.getInt("salary"));
return emp;
```

```
}
```

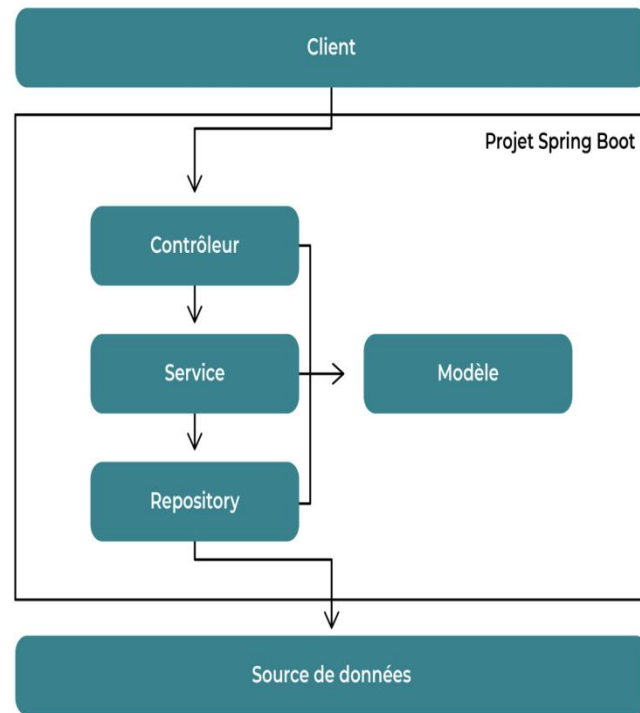
```
}
```

```
}
```

9. Structure d'une API REST

Les applications Spring Boot peuvent être découpées en couches :

- **couche Controller** : gestion des interactions entre l'utilisateur de l'application et l'application ;
- **couche Service** : implémentation des traitements métiers spécifiques à l'application ;
- **couche Repository** : interaction avec les sources de données externes ;
- **couche Model** : implémentation des objets métiers qui seront manipulés par les autres couches.



Représentation visuelle de l'architecture en couches

1. Création d'une API REST avec JdbcTemplate

Configuration des sources de données

application.properties (par défaut)

```
spring.application.name=demo
server.port = 8080
spring.datasource.url=jdbc:mysql://localhost:3306/
base
spring.datasource.username=root
spring.datasource.password= root
#spring.datasource.driver-class-
name=com.mysql.cj.jdbc.Driver
spring.datasource.driver-class-
name=com.mysql.jdbc.Driver
```

databases.properties

```
jdbc.url=jdbc:mysql://localhost:3306/base
jdbc.username=root
jdbc.password= root
jdbc.driverClassName=com.mysql.jdbc.Driver
```

En java

```
package com.example.demo;
import javax.sql.DataSource;
import
org.springframework.beans.factory.annotation.Auto
wired;
```



```

import
org.springframework.boot.jdbc.DataSourceBuilder
;
import
org.springframework.context.annotation.Bean;
import
org.springframework.context.annotation.Configuration;
import
org.springframework.context.annotation.PropertySource;
import
org.springframework.core.env.Environment;
import
org.springframework.jdbc.core.JdbcTemplate;
import
org.springframework.jdbc.datasource.DriverManagerDataSource;

@Configuration
// @PropertySource(value = {
// "classpath:database.properties" })
public class AppConfig {
// Methode 1
    @Autowired
    private Environment env;

    @Bean
    public DataSource dataSource()
    {
        // Lire fichier databases.properties
        DriverManagerDataSource dataSource =
        new DriverManagerDataSource();

        dataSource.setDriverClassName(env.getProperty(
        "jdbc.driverClassName"));

        dataSource.setUrl(env.getProperty("jdbc.url"));

        dataSource.setUsername(env.getProperty("jdbc.u
        sername"));

        dataSource.setPassword(env.getProperty("jdbc.p
        assword"));
        return dataSource;
    }

//Methode 2
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource =
        new DriverManagerDataSource();

        dataSource.setDriverClassName("com.mysql.jdbc
        .Driver");

```

```

        dataSource.setUrl("jdbc:mysql://localhost:3306/ba
        se");
        dataSource.setUsername("root");
        dataSource.setPassword("root");

        return dataSource;
    }

// Methode 3
    @Bean
    public DataSource dataSource() {
        DataSourceBuilder dataSourceBuilder =
        DataSourceBuilder.create();

        dataSourceBuilder.driverClassName("com.mysql.j
        dbc.Driver");

        dataSourceBuilder.url("jdbc:mysql://localhost:3306
        /mysql");
        dataSourceBuilder.username("root");
        dataSourceBuilder.password("root");
        return dataSourceBuilder.build();
    }

    @Bean

        public EmployeeDAO employeeBean()
        {

            // Returns the class object
            return new EmployeeDAOImpl(dataSource()
            );
        }

        @Bean

            public EmployeeDAO employeeBean2()
            {

                // Returns the class object
                return new EmployeeDAOImpl(dataSource()
                );
            }
        }
    }

```

Le modele

1. Classe Employee

```

package com.example.demo.model;
public class Employee{
    private int id;
    private String name;
    private int salary;

    public Employee () {

```

```

    }

    public Employee (int id, String
name, int salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getSalary() {
        return salary;
    }

    public void setSalary(int salary) {
        this.salary = salary;
    }

    @Override
    public String toString() {
        return "Employee{" + "id=" +
id + ", name=" + name + ", salary=" +
salary
                                + "}";
    }
}

```

2. Le DAO (service)

2.1. Interface

```

package com.example.demo.dao;

import java.util.List;

import
com.example.demo.model.Employee;
public interface EmployeeDAO {

    public List<Employee> findAll();

    public Employee findById(int id);

```

```

    public int deleteById(int id);

    public int save(Employee e);

    public int update(Employee e, int id)

    public void create();
    public int count();

}

```

2.2. Implementation

```

package com.example.demo;
import java.util.List;
import javax.sql.DataSource;
import
org.springframework.beans.factory.annotat
ion.Autowired;
import
org.springframework.jdbc.core.BeanProper
tyRowMapper;
import
org.springframework.jdbc.core.JdbcTempl
ate;
import
org.springframework.stereotype.Compone
nt;
import
org.springframework.stereotype.Repositor
y;
import
org.springframework.jdbc.core.PreparedSt
atementCallback;

import java.sql.PreparedStatement;
import java.sql.SQLException;
import
org.springframework.dao.DataAccessExce
ption;
import
org.springframework.jdbc.core.RowMappe
r;
import java.sql.ResultSet;
import
com.example.demo.model.Employee;
//@Repository
public class EmployeeDAOImpl
implements EmployeeDAO{

    // @Autowired

    //private JdbcTemplate
jdbcTemplate;

```

```

JdbcTemplate jdbcTemplate;

@Autowired
public
EmployeeDAOImpl(DataSource
dataSource) {
jdbcTemplate = new
JdbcTemplate(dataSource);
}

@Override
public void create()
{
    System.out.println("Creating tables");
    jdbcTemplate.execute("drop table if
exists customers");
    jdbcTemplate.execute("create table
customers(id int, first_name varchar(255),
last_name varchar(255))");
}

@Override
public int count() {
    return
jdbcTemplate.queryForObject("select
count(*) from employee", Integer.class);
}

@Override
public List<Employee> findAll() {
    //OK return
jdbcTemplate.query("SELECT * FROM
employee", new
BeanPropertyRowMapper<Employee>(Em
ployee.class)); */
    return jdbcTemplate.query("SELECT *
FROM employee ", new
EmployeeMapper());

    // log.info("List Employees : "+empList);
}

@Override
public Employee findById(int id) {
    return
jdbcTemplate.queryForObject("SELECT *
FROM employee WHERE id=?", new
BeanPropertyRowMapper<Employee>(Em
ployee.class), id);
}

```

```

@Override
public int deleteById(int id) {
    return
jdbcTemplate.update("DELETE FROM
employee WHERE id=?", id);
}

@Override
public int save(Employee e) {
    return
jdbcTemplate.update("INSERT INTO
employee values
(?,?,?)",e.getId(),e.getName(),
e.getSalary() );
}

@Override
public int update(Employee e, int id)
{

return jdbcTemplate.update("UPDATE
employee SET name = ?, salary = ?
WHERE id = ?", e.getName(),
e.getSalary(), id);
}

private static class
EmployeeMapper implements RowMapper
<Employee>{
    public Employee
mapRow(ResultSet rs, int rowNum) throws
SQLException {
        Employee emp = new
Employee();
        emp.setId(rs.getInt("id"));

        emp.setName(rs.getString("name"));

        emp.setSalary(rs.getInt("salary"));
        return emp;
    }
}

```

3. Controlleur

```
package com.example.demo;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@RestController
public class EmployeeController {

    @Autowired
    private EmployeeDAO eDAO;

    @GetMapping("/essai")
    public String hello() {
        return "SALAMA RABE";
    }

    @RequestMapping(value = "/employees",
        method = RequestMethod.GET)

        // @GetMapping("/employees")
```

```
public List<Employee> findAll() {
    return eDAO.findAll();
}

@RequestMapping(value =
    "/employees/{id}", method =
    RequestMethod.GET)
    // @GetMapping("/employees/{id}")
    public Employee
    findById(@PathVariable int id) {
        return eDAO.findById(id);
    }

    @DeleteMapping("/employees/{id}")
    public String
    deleteById(@PathVariable int id) {
        return
        eDAO.deleteById(id)+" Employee(s) delete
        from the database";
    }

    @RequestMapping(value =
    "/employees", method =
    RequestMethod.POST)
    // @PostMapping("/employees")
    public String save(@RequestBody
    Employee e) {
        return eDAO.save(e)+"
        Employee(s) saved successfully";
    }

    @PutMapping("/employees/{id}")
    public String
    update(@RequestBody Employee e,
    @PathVariable int id) {
        return eDAO.update(e, id)+"
        Employee(s) updated successfully";
    }
}
```

2. Création d'une API REST avec JPA

Exemple de Spring Boot RESTful CRUD avec base de données MySQL et JPA en utilisant une interface **CrudRepository**

Les données Spring Boot fournissent une interface **CrudRepository** pour les opérations CRUD génériques dans le package **org.springframework.data.repository**. Pour créer une connexion avec la base de données MySQL, nous devons configurer les propriétés de la source de données dans le

fichier `application.properties` commençant par `spring.datasource.*`. Spring Boot utilise `spring-boot-starter-data-jpa` pour configurer l'API Spring Java Persistence (JPA).

Dans cet exemple, nous allons créer une application Spring Boot qui se connecte à notre base de données MySQL externe, consommera et produira les données **JSON** et effectuera les opérations suivantes :

- Enregistrez les données soumises par l'utilisateur dans la base de données.
- Récupérer toutes les données soumises par les utilisateurs à partir de la base de données
- Récupérez des données particulières de la base de données par un identifiant donné.
- Mettre à jour les données existantes.
- Et supprimez un enregistrement de la base de données.

1. Interface du référentiel (repository) Crud

CrudRepository est une interface fournie par Spring Framework lui-même. **CrudRepository** étend Spring Data **Repository** qui est une interface centrale de marqueur de référentiel (repository). **CrudRepository** fournit la méthode générique pour les opérations de création, de lecture, de mise à jour et de suppression (CRUD).

public interface **CrudRepository**<T,ID> extends **Repository**<T,ID>

où T = nom de l'entité et ID correspond au Type de l' id de l'entité géré par le repository (en général @Id)

- **T** : type de domaine géré par le référentiel (généralement le nom de la classe Entité/Modèle)
- **ID** : Type de l'identifiant de l'entité gérée par le référentiel (généralement la classe wrapper de votre @Id qui est créée dans la classe Entity/Model)

CrudRepository contient au total 11 méthodes pour le fonctionnement CRUD, certaines d'entre elles sont répertoriées ci-dessous et que nous utiliserons dans cette application :

- **<S extends T> S save(S entity)**: Enregistrez et mettez à jour une entité donnée. L'entité ne peut pas être nulle et l'entité enregistrée à retourner ne sera jamais nulle.
- **Iterable<T> findAll()**: Renvoie toutes les entités.

- **Optional<T> findById(ID id)**: Récupère une entité par son ID. L'ID ne peut pas être nul.
- **void deleteById(ID id)**: Supprime l'entité avec l'ID donné. L'ID ne peut pas être nul.
- **long count()** retourne le nombre d'entités

2. Technologies utilisées

1. Suite d'outils Spring 4
2. JDK8
3. Maven3
4. Spring-boot 2.1.2.RELEASE
5. Base de données MySQL

3. Schéma de base de données

Voici la structure des tables de la base de données MySQL utilisée dans cet exemple.

```
CREATE TABLE `country_master` (
  `country_id` int(4) AUTO_INCREMENT,
  `country_name` varchar(20),
  `country_lang` varchar(10),
  `country_population` int(5),
  PRIMARY KEY (`country_id`));
```

4. Dépendances requises

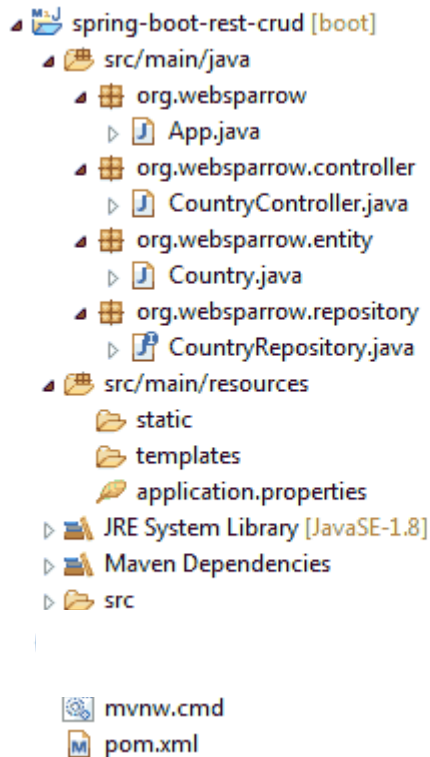
Pour créer une application Spring Boot RESTful CRUD, vous avez besoin des dépendances suivantes.

pom.xml

```
<dependencies>
  <!-- Spring boot data -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <!-- spring boot web -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!-- MySQL database connector -->
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```


5. Structure du projet

La structure finale du projet de notre application dans STS ide ressemblera à ceci.



6. application.properties

Configurez la source de données, les propriétés JPA, etc. dans le **fichier application.properties**.

Ces propriétés sont automatiquement lues par Spring Boot.

application.properties

```
# MySQL database connecting strings
spring.datasource.url=jdbc:mysql://localhost:3306/
websparrow
spring.datasource.username=root
spring.datasource.password=root
#spring.datasource.driver-class-
name=com.mysql.cj.jdbc.Driver
spring.datasource.driver-class-
name=com.mysql.jdbc.Driver
```

```
# JPA property settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.show_sql=true
```

```
spring.jpa.database-
platform=org.hibernate.dialect.MySQL5Dialect
spring.jpa.generate-ddl=true
spring.jpa.properties.hibernate.show_sql=true
spring.jpa.show-sql=true
```

On peut configurer en java :

JPAConfig.java

```
package com.example.demo;

import javax.persistence.EntityManagerFactory;
import javax.sql.DataSource;
import java.util.HashMap;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.jdbc.datasource.embedded.EmbeddedDatabaseBuilder;
import org.springframework.jdbc.datasource.embedded.EmbeddedDatabaseType;
import org.springframework.orm.jpa.JpaTransactionManager;
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
import org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.jdbc.DataSourceBuilder;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.PropertySource;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

@Configuration
@EnableJpaRepositories(
```

```

    basePackages =
    "com.example.demo.repository",
    entityManagerFactoryRef =
    "userEntityManager",
    transactionManagerRef =
    "userTransactionManager"
)
public class JPAConfig {
    @Bean
    public
    LocalContainerEntityManagerFactoryBean
    userEntityManager() {
        LocalContainerEntityManagerFactoryBean em
        = new
        LocalContainerEntityManagerFactoryBean();
        em.setDataSource(userDataSource());
        em.setPackagesToScan(
            new String[] { "com.example.demo.model"
        });

        HibernateJpaVendorAdapter vendorAdapter
        = new HibernateJpaVendorAdapter();
        em.setJpaVendorAdapter(vendorAdapter);
        HashMap<String, Object> properties = new
        HashMap<>();
        properties.put("hibernate.hbm2ddl.auto",
            env.getProperty("hibernate.hbm2ddl.auto"));
        properties.put("hibernate.dialect",
            env.getProperty("hibernate.dialect"));
        em.setJpaPropertyMap(properties);

        return em;
    }

    @Bean
    public DataSource userDataSource() {

        DriverManagerDataSource dataSource = new
        DriverManagerDataSource();

        dataSource.setDriverClassName("com.mysql.jdbc
        .Driver");

        dataSource.setUrl("jdbc:mysql://localhost/base");
        dataSource.setUsername("root");
        dataSource.setPassword("root");
        return dataSource;
    }

    @Bean
    public PlatformTransactionManager
    userTransactionManager() {

        JpaTransactionManager transactionManager
        = new JpaTransactionManager();
    }

```

```

        userEntityManager().getObject());
        return transactionManager;
    }
}

```

7. Créer l'entité

Créez une classe de modèle **Country**, définissez ses attributs et annotez avec **@Entity** une annotation **@Table** en haut de la classe. L'annotation **@Table** est utilisée pour mapper votre table de base de données existante avec cette classe et **@Column** les colonnes de la table de mappage d'annotations.

Remarque : Si la table n'est pas disponible dans votre base de données, l'annotation **@Entity** indique à Hibernate de créer une table à partir de cette classe.

Country.java

```

package org.websparrow.entity;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name = "country_master")
public class Country {

    // TODO: Generate getters and setters...

    @Id
    @GeneratedValue(strategy=Generation
    Type.AUTO)
    @Column(name = "country_id")
    private int countryId;

    @Column(name = "country_name")
    private String countryName;

    @Column(name = "country_lang")
    private String countryLang;

    @Column(name = "country_population")
    private int countryPopulation;

}

```

8. Créez le référentiel (Repository)

Créez une interface **CountryRepository** qui étend **CrudRepository**. Cela sera AUTO IMPLÉMENTÉ par Spring dans un Bean appelé **countryRepository**.

CountryRepository.java

```
package org.websparrow.repository;

import org.springframework.data.repository.CrudRepository;
import org.websparrow.entity.Country;

public interface CountryRepository extends
    CrudRepository<Country, Integer> {

}
```

9. Créez le contrôleur

Créez une classe **CountryController** qui gère la demande de l'utilisateur pour effectuer des opérations de création, de lecture, de mise à jour et de suppression.

CountryController.java

```
package org.websparrow.controller;

import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
```

```
import org.springframework.web.bind.annotation.RestController;
import org.websparrow.entity.Country;
import org.websparrow.repository.CountryRepository;

@RestController
@RequestMapping("/country")
public class CountryController {

    @Autowired
    CountryRepository countryRepository;
    // insert new country into database
    @PostMapping("/add")
    public Country
    addCountry(@RequestBody Country country) {
        return countryRepository.save(country);
    }

    // fetch all country list from database
    @GetMapping("/all")
    public Iterable<Country> allCountry() {

        return countryRepository.findAll();
    }

    // fetch specific country by their ID
    @GetMapping("/{countryId}")
    public Optional<Country>
    countryById(@PathVariable("countryId") int
        countryId) {

        return countryRepository.findById(countryId);
    }

    // update existing country
    @PutMapping("/update")
    public Country
    updateCountry(@RequestBody Country country) {

        return countryRepository.save(country);
    }

    // delete country from database
    @DeleteMapping("/{countryId}")
    public void
    deleteCountry(@PathVariable("countryId") int
        countryId) {
        countryRepository.deleteById(countryId);
    }
}
```

10. Rendre l'application exécutable

Créez une **App** classe et exécutez-la.

```
package org.websparrow;

import org.springframework.boot.
SpringApplication;
import org.springframework.boot.autoconfigure.
SpringBootApplication;

@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

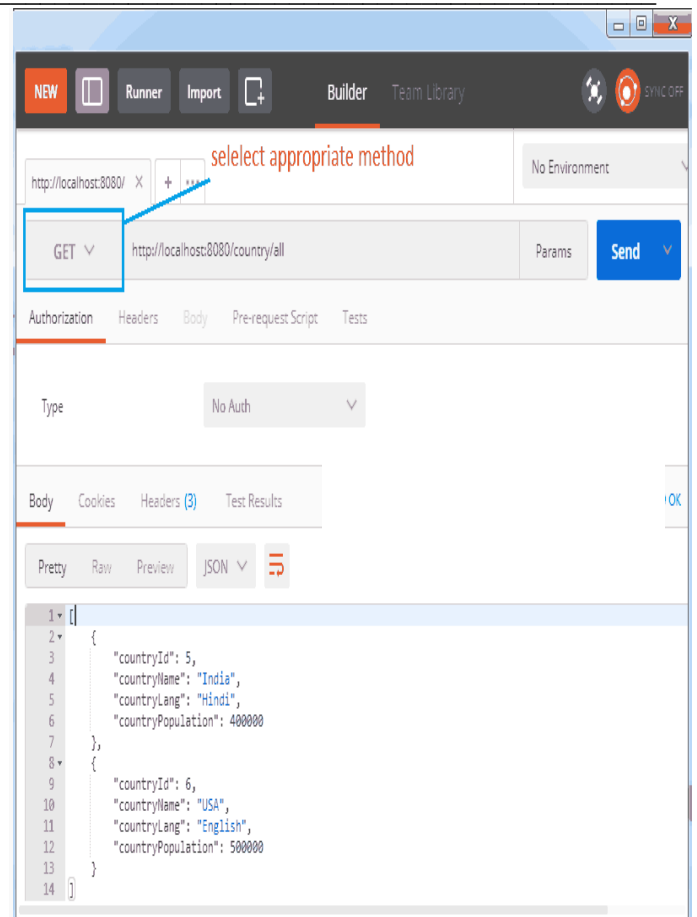
11. Testez l'application

Pour tester l'application, ouvrez [Postman](#) et suivez les étapes ci-dessous :

1. Pour ajouter un nouveau pays, utilisez l'URL `http://localhost:8080/country/add`, sélectionnez la méthode **POST**, définie dans l'onglet **En-têtes (headers)**, sélectionnez **raw** dans l'onglet **Corps (body)** et collez le code suivant. **Content-Type="application/json"**

```
{
  "countryName": "India",
  "countryLang": "Hindi",
  "countryPopulation": 400000
}
```

2. Pour récupérer la liste de tous les pays, utilisez l'URL `http://localhost:8080/country/all` avec la requête **GET**.



3. De même, vous pouvez effectuer l'opération de mise à jour et de suppression. Pour la mise à jour, utilisez **PUT** et supprimez, utilisez la requête **DELETE**.

Lien: https://websparrow.org/spring/spring-boot-restful-crud-example-with-mysql-database#google_vignette

Exemple 3 : Spring Boot RESTful CRUD avec base de données MySQL et JPA en utilisant une interface **JpaRepository**

La création d'opérations CRUD avec MySQL dans Spring Boot nécessite l'intégration de Spring Data JPA et du pilote MySQL JDBC.

Créez 4 packages et créez des classes et des interfaces à l'intérieur de ces packages :

- entity
- repository
- service
- controller

Voici un guide étape par étape :

1. Configurer les dépendances

Ajoutez Spring Data JPA et le pilote MySQL à votre pom.xml :

```
<!-- Spring Data JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-
jpa</artifactId>
</dependency>
```

```
<!-- MySQL JDBC driver -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
</dependency>
```

2. Configurer la base de données

Spécifiez vos propriétés MySQL dans application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/
mydatabase?useSSL=false
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-
name=com.mysql.jdbc.Driver
# JPA property settings
spring.jpa.database-
platform=org.hibernate.dialect.MySQL5Dialect
spring.jpa.generate-ddl=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.show_sql=true
spring.jpa.show-sql=true
```

3. Créer une entité

Définissez votre entité JPA :

```
@Entity
public class User {
  @Id
  @GeneratedValue(strategy =
GenerationType.IDENTITY)
  private Long id;
  private String name;
  private String email;

  // Getters, Setters, Constructors, hashCode,
equals, toString
}
```

4. Créer un référentiel (Repository)

Dépôt Jpa (JpaRepository)

JpaRepository est une extension spécifique **JPA (Java Persistence API)** de Repository. Il contient l'API complète de **CrudRepository** et **PagingAndSortingRepository**. Il contient donc

une API pour les opérations CRUD de base ainsi qu'une API pour la pagination et le tri. Par exemple, il contient flush(), saveAndFlush(), saveAllAndFlush(), deleteInBatch(), etc. ainsi que les méthodes disponibles dans CrudRepository

Syntaxe:

```
public interface JpaRepository<T,ID>
extends PagingAndSortingRepository<T,ID>,
QueryByExampleExecutor<T>
```

Où:

- **T** : type de domaine géré par le référentiel (généralement le nom de la classe Entité/Modèle)
- **ID** : Type de l'identifiant de l'entité gérée par le référentiel (généralement la classe wrapper de votre @Id qui est créée dans la classe Entity/Model)

Illustration:

```
public interface DepartmentRepository extends
JpaRepository<Department, Long> {}
```

Méthodes

Certaines des méthodes les plus importantes disponibles dans JpaRepository sont indiquées ci-dessous.

Méthode 1 : saveAll() : enregistre toutes les entités données.

Syntaxe: <S extends T> List<S> saveAll(Iterable<S> entities)

Méthode 2 : getById() : renvoie une référence à l'entité avec l'identifiant donné.

Syntaxe: T getById (ID id)

Méthode 3 : flush() : vide toutes les modifications en attente dans la base de données.

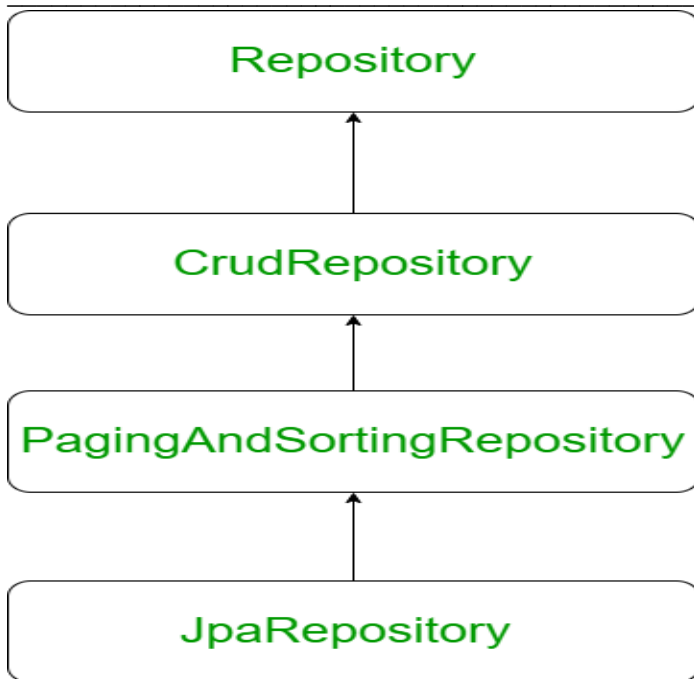
Syntaxe: void flush()

Méthode 4 : saveAndFlush() : enregistre et retourne une entité et supprime les modifications instantanément.

Syntaxe: <S extends T> S saveAndFlush(S entity)

Méthode 5 : deleteAllInBatch() : supprime les entités données dans un lot,

Syntaxe: void deleteAllInBatch(Iterable<T> entities)



Spring Data JPA fournit le package JpaRepository pour simplifier les opérations CRUD :

```
public interface UserRepository extends
JpaRepository<User, Long> {}
```

5. Créer un service

Voici une couche de service simple :

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    public User getUserById(Long id) {
        return
userRepository.findById(id).orElse(null);
    }

    public User createUser(User user) {
        return userRepository.save(user);
    }

    public User updateUser(Long id, User user) {
        if (userRepository.existsById(id)) {
            user.setId(id);
            return userRepository.save(user);
        } else {
            return null;
        }
    }
}
```

```
public void deleteUser(Long id) {
    userRepository.deleteById(id);
}
}
```

6. Créer un contrôleur

Une API RESTful pour les opérations CRUD :

```
@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }

    @GetMapping("/{id}")
    public User getUserById(@PathVariable Long
id) {
        return userService.getUserById(id);
    }

    @PostMapping
    public User createUser(@RequestBody User
user) {
        return userService.createUser(user);
    }

    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long
id, @RequestBody User user) {
        return userService.updateUser(id, user);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void>
deleteUser(@PathVariable Long id) {
        userService.deleteUser(id);
        return ResponseEntity.noContent().build();
    }
}
```

Lien :

<https://www.iditect.com/programming/spring-boot/spring-boot-crud-operations-using-mysql-database.html>